

Estudo detalhado da Xclipse 940 — evidências de runtime, Device Tree e rotas de desenvolvimento

Projeto interno: XO940

Hardware analisado: Samsung Xclipse 940 no dispositivo SM-S721B

Data da revisão: 2026-09-06

Autor: Manus AI

Conclusão executiva

O pacote novo acrescenta evidência substancial sobre o caminho real da Xclipse 940. A coleta de runtime expõe um nó `sgpu@22200000` no Device Tree, um domínio de energia G3DCORE, regiões de memória reservada, DMA heap de GPU, caminhos de IOMMU/SysMMU e propriedades de DVFS, IRQ e registradores. Esse conjunto é útil para portar a descrição de plataforma e para entender o contrato necessário antes de qualquer driver de espaço de usuário.

Os traces P4.5, P4.6 e P4.12 são a evidência mais importante desta rodada. Eles mostram chamadas `amdgpu_cs_ioctl`, execução pelo scheduler em `gfx_0.0.0` e `amdgpu_ib_schedule`, incluindo `num_ibs=3`. P4.5 também mostra o vínculo temporal entre `sgpu_pio_map_queue`, `amdgpu_vm_bo_cs`, `amdgpu_vm_flush` e o envio do job. Isso confirma atividade de submissão GFX no caminho observado.

A evidência ainda não demonstra que um novo driver independente consegue criar e executar um workload controlado. P5.04 não capturou um IB compute nem eventos de dispatch. A presença de exports OpenCL, de `libSGPU0openCL.so` e de símbolos de compilador prova uma superfície instalada e um backend proprietário existente, mas não prova que qualquer programa externo possa usar essa superfície fora dos namespaces e permissões originais.

A rota tecnicamente mais promissora é incremental. Primeiro, deve-se reproduzir o contrato de plataforma e memória com base no Device Tree e na UAPI SGPU. Depois, deve-se correlacionar submissão, VM, fences e conclusão em uma aplicação controlada. Somente após isso faz sentido estudar a tradução de comandos, ISA e uma camada Vulkan.

1. Regra de evidência

Os nomes dos arquivos e os títulos das etapas não foram tratados como resultados. Cada afirmação foi classificada por sua observação concreta:

Classe	Significado	Uso nesta revisão
Observado	aparece em saída de runtime, trace, inventário ELF ou Device Tree	pode sustentar uma conclusão factual limitada
Corroborado	aparece em mais de uma coleta independente	pode orientar uma prioridade de implementação
Indício	símbolo, caminho, string ou estrutura sem execução controlada	orienta investigação, não prova capacidade
Não demonstrado	o arquivo foi executado, mas a saída não contém a operação procurada	deve bloquear conclusões otimistas
Roteiro	instrução, inicializador, comando ou preparação	não é resultado e não deve ser apresentado como teste bem-sucedido

P4.04, P5.04 e arquivos de descoberta de tracepoints foram tratados com cuidado. A existência de seções “TRACE START” e “TRACE RESULT” não significa que o workload-alvo tenha sido executado. Em particular, P5.04 possui apenas atividade de `tracing_mark_write` e não contém `amdgpu_cs_ioctl`, `amdgpu_sched_run_job`, `amdgpu_ib_schedule`, `dispatch` ou `readback`.

2. Device Tree e plataforma

O mapa de runtime identifica o nó principal em `/sys/firmware/devicetree/base/sgpu@22200000`. O compatible observado é `samsung-sgpu,samsung-sgpu`. O nó possui os filhos `gpu_pm`, `gpu_doorbell`, `gpu_debug`, `gpu_smntarg` e `gpu_sysreg`. Os nomes de região observados são `gpu`, `doorbell`, `debug`, `pwrctl`, `sysreg` e `htu`.

As interrupções são nomeadas `SGPU` e `GPU-AFM`. O nó referencia o phandle `0xc3`, que corresponde ao domínio `pd_g3dcore@0 / pd_g3dcore`. O domínio foi observado como `samsung,exynos-pd` com estado `okay`. Essa relação é importante porque inicializar a GPU sem o domínio G3DCORE, sem clocks e sem reset coerentes pode produzir falhas que parecem de MMU ou de comandos, mas são de plataforma.

A ramificação técnica publicada separa o caminho da GPU dos demais consumidores do SoC. Ela inclui o nó SGPU, G3D, BTS G3D, memória reservada, DMA heap, grupos IOMMU e vínculos térmicos. Ela não é a árvore do repositório e não inclui a documentação do projeto.

3. Memória, VM e IOMMU

A coleta P2.10 registra 119.929 eventos `amdgpu_vm_pte_pde`, 11.307 `amdgpu_vm_set_ptes`, 10.998 `amdgpu_vm_update_ptes`, 1.947 `amdgpu_vm_flush`, 262 criações de BO e 262 movimentos de BO. Isso é evidência forte de que o caminho observado realiza gerenciamento de objetos e atualização de tabelas de página durante atividade gráfica.

P4.5 e P4.6 ligam a memória à submissão. Antes de alguns jobs aparecem `sgpu_pio_map_queue_entry` e `sgpu_pio_map_queue_exit`; durante o job aparecem `amdgpu_vm_bo_cs` e `amdgpu_vm_flush`; em

seguida aparecem `amdgpu_sched_run_job` e `amdgpu_ib_schedule`. Essa sequência é um mapa operacional melhor do que uma lista isolada de símbolos.

A existência de endereços de page directory, PTEs e flags nos traces não revela, por si só, o formato completo das tabelas, a política de cache, a semântica de todos os bits ou o mecanismo de recuperação de fault. Esses itens continuam como trabalho de engenharia reversa controlada.

4. Submissão GFX, scheduler e fences

P4.12 contém uma janela com 13 chamadas `amdgpu_cs_ioctl`, 13 `amdgpu_sched_run_job` e 13 `amdgpu_ib_schedule`. Os eventos usam `timeline=gfx_0.0.0`, `ring_name=gfx_0.0.0` e `num_ibs=3`. P4.6 confirma a mesma cadeia junto com `amdgpu_vm_bo_cs`, `amdgpu_vm_flush`, `amdgpu_ib_pipe_sync` e `sgpu_job_dependency` habilitados.

O que está comprovado é a passagem de jobs gráficos pelo caminho de submissão do driver observado. O que ainda não está comprovado é a leitura direta do conteúdo dos IBs, a interpretação dos pacotes pelo hardware, o sinal de fence no mesmo evento e o resultado de um buffer controlado. Os eventos clássicos de fence `amdgpu_fence*` estavam indisponíveis em P4.12; portanto, não se deve escrever “fence concluída” somente porque um job foi agendado.

A próxima coleta deve correlacionar o mesmo `sched_job`, `context` e `seqno` entre `ioctl`, scheduler, IB, DMA fence e evento de conclusão. Se o kernel não expuser todos os eventos, deve-se usar uma combinação documentada de tracepoints disponíveis, estado de ring e um buffer de resultado com assinatura conhecida.

5. Caminho Android, Vulkan e OpenCL

O inventário ELF confirma a presença de `vulkan.samsung.so`, `libdrm_sgpu.so`, `libOpenCL.so` e `libSGPUOpenCL.so` no ambiente fornecido. A cadeia de runtime também mostra processos de câmera, `surfaceflinger` e bibliotecas vendor carregadas em namespaces Android específicos. Isso confirma que o firmware da Samsung possui consumidores reais da GPU e uma pilha proprietária funcional em seus próprios limites.

A presença de exports OpenCL como `clEnqueueNDRangeKernel`, `clEnqueueReadBuffer`, `clBuildProgram` e `clCreateProgramWithIL` é uma oportunidade de estudo. Ela não prova que o backend possa ser reutilizado por um novo loader, porque símbolos exportados podem depender de inicialização privada, propriedades de contexto, permissões SELinux, bibliotecas auxiliares e formatos internos.

P5.01 identifica bibliotecas OpenCL e seus exports, mas não fecha um dispatch compute independente. P5.04 não contém o dispatch procurado. A conclusão correta é: **há uma superfície OpenCL proprietária instalada e há uma rota de investigação; a execução compute controlada ainda não foi demonstrada nesta coleta.**

6. Cinco prioridades críticas

ID	Prioridade	Por que é crítica	Critério de saída
C1	Reproduzir o contrato Device Tree e power/clock/reset	sem plataforma estável, os demais resultados ficam ambíguos	mapa de recursos, IRQ, power-domain, clocks e reset reproduzido em documentação e probe mínimo
C2	Fechar memória, IOMMU e VM	todo driver precisa alocar, mapear, sincronizar e desfazer BOs corretamente	sequência controlada de BO → map → VM update → flush → unmap com erro e limpeza observáveis
C3	Fechar a submissão GFX e a conclusão	já há evidência de scheduling, mas falta um workload controlado com conclusão inequívoca	job próprio com IB conhecido, fence correlacionada e buffer de saída validado
C4	Identificar o contrato UAPI/ABI mínimo	símbolos e paths não substituem ioctl e estruturas compatíveis	tabela de ioctls, structs, flags, alinhamentos e permissões necessários para um cliente mínimo
C5	Obter um caminho compute controlado	OpenCL vendor indica oportunidade, mas P5.04 não executou compute	dispatch simples com entrada/saída conhecida, ou prova objetiva do bloqueio e de sua camada

7. Cinco descobertas rápidas

ID	Descoberta rápida	Evidência já disponível	Utilidade
Q1	Enumerar exatamente os <code>reg-names</code> e os intervalos do nó SGPU	R1 lista <code>gpu</code> , <code>doorbell</code> , <code>debug</code> , <code>pwrctl</code> , <code>sysreg</code> e <code>htu</code>	reduz erros no primeiro mapa de MMIO
Q2	Correlacionar os três <code>num_ibs=3</code> com contexto e sequência	P4.12 já fornece <code>sched_job</code> , <code>context</code> , <code>seqno</code> e <code>ring</code>	cria um identificador de job para as próximas coletas
Q3	Catalogar os tracepoints SGPU disponíveis	P5.05 lista eventos de PIO, <code>devfreq</code> , IFPO e dependências	permite capturas menores e mais informativas
Q4	Mapear namespaces e dependências do OpenCL vendor	P3.9.x e R5 listam bibliotecas, SONAME e NEEDED	revela onde uma tentativa externa seria bloqueada
Q5	Comparar snapshots de VM por PID e por workload	P2.09, P2.10 e P2.11 já separam VM, PTE e PID	diferencia atividade de <code>RenderThread</code> , câmera e compositor

8. Falhas e rotas de investigação

A principal falha observável não é uma ausência de GPU. É a ausência de uma fronteira pública e independente entre a pilha proprietária e um cliente novo. O Android possui nós DRM, bibliotecas vendor, namespaces e políticas de segurança, mas os relatórios R2 não identificam um caminho de cliente genérico com todos os atributos disponíveis.

A segunda falha é metodológica: muitos traces mostram atividade real do sistema, porém o workload não é isolado. `RenderThread`, `GrallocUploadTh`, Chrome, câmera e compositor podem gerar jobs simultâneos. Sem PID, context e seqno controlados, um evento real pode ser atribuído à operação errada.

A terceira falha é a falta de um resultado compute controlado. A rota OpenCL deve ser estudada como camada de observação e como referência de comportamento, não como API automaticamente reutilizável. A investigação deve começar por um programa mínimo que identifique plataforma, dispositivo, contexto, fila, kernel e leitura de um buffer; cada fase precisa de um marcador e de uma saída verificável.

9. O que não foi publicado

O repositório não publica os testes, scripts de captura, traces crus, dumps de memória, bibliotecas `.so`, binários ELF, ZIPs internos, código proprietário ou arquivos que permitam baixar diretamente o material bruto do dispositivo. O GitHub recebe apenas documentação derivada, hashes quando necessários, inventários sanitizados, mapas técnicos, relatórios de estudo e PDFs.

10. Referências internas de evidência

Os identificadores abaixo apontam para os artefatos recebidos na análise local. Os artefatos crus permanecem fora do repositório público, mas os nomes são preservados para auditoria privada.

ID	Artefato analisado	Tipo de evidência
E1	R1_Device_Tree_SGPU.txt , R1_power_iommu.txt , R1_g3d_links.txt	Device Tree e plataforma
E2	R2_Runtime_Inventory_MASTER.txt	inventário runtime e nós DRM
E3	R3_MASTER.txt , R5_ICD_ELF_INVENTORY.txt	bibliotecas, ELF e dependências
E4	P2_10_RESUM0.txt , P2_11_PID_VM_CORRELATION.txt	VM, PTE, BO e PID
E5	P4_5_SGPU_LIVE_JOB_SUBMISSION.txt , P4_6_SGPU_JOB_COMPLETION_FENCE_TRACE.txt , P4_12_SGPU_FENCE_SIGNAL_JOB_RETIRE_RING_RPTR_CORRELATION.txt	submissão, scheduler e IB
E6	P5_01_COMPUTE_PATH.txt , P5_04_COMPUTE_IB_CAPTURE.txt , P5_05_TRACEPOINT_DISCOVERY.txt	compute, negativa de captura e observabilidade
E7	P3_9.34_FINAL_RUNTIME_CHAIN.txt	cadeia Android e namespaces

Referências

13

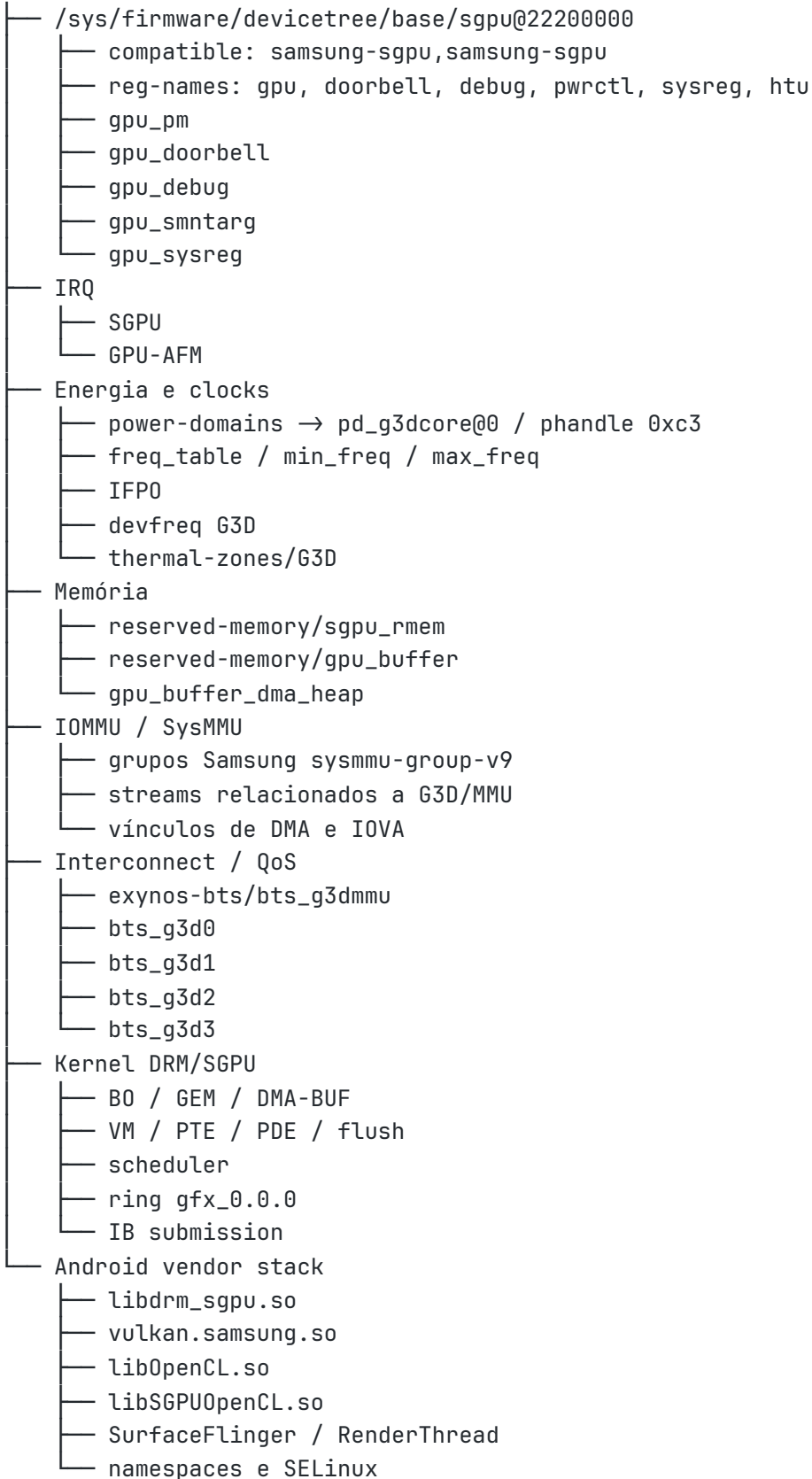
Este estudo descreve evidências fornecidas pelo usuário e não afirma compatibilidade, segurança ou funcionalidade de um driver ainda não implementado.

Ramificação técnica da Xclipse 940

Esta é a **ramificação técnica** do hardware, não a árvore de arquivos do projeto. Ela representa os caminhos observados no Device Tree, kernel, GPU, memória, energia, IOMMU e Android.

SoC Exynos / plataforma SM-S721B

└─ G3D / Xclipse 940



Caminho operacional observado

```
RenderThread / Gralloc
→ sgpu_pio_map_queue
→ amdgpu_vm_bo_cs
→ amdgpu_vm_flush
→ amdgpu_cs_ioctl
→ amdgpu_sched_run_job
→ amdgpu_ib_schedule
→ ring gfx_0.0.0
→ eventos de limpeza/unmap
```

O mapa mostra relações e pontos de observação. Não transforma nomes de nós em uma especificação completa do hardware. Endereços, flags e formatos devem ser validados por coletas controladas e pela fonte de kernel licenciada.

Próximo refinamento

O próximo mapa deve adicionar, para cada recurso, a origem do phandle, a unidade de clock/reset, a relação IOMMU/stream e o mecanismo de erro. O objetivo é produzir uma descrição reproduzível antes de escrever um cliente de GPU.

Mapa público de evidências — coleta de 2026-09-06

Este mapa conecta famílias de artefatos a conclusões permitidas. Ele não redistribui os arquivos crus.

Família	Observação	Conclusão permitida	Não concluir
R1	<code>sgpu@22200000</code> , filhos SGPU, IRQs, phandle G3DCORE, memória e IOMMU	a plataforma e seus vínculos foram observados	que todos os registradores já estejam decodificados
R2	nós <code>/dev/dri/card*</code> e <code>renderD*</code> e permissões	existem nós DRM acessíveis no snapshot	qual nó é o alvo sem identificação adicional
R3/R5	<code>vulkan.samsung.so</code> , <code>libdrm_sgpu.so</code> , OpenCL e SONAME/NEEDED	a pilha vendor está instalada	que bibliotecas vendor sejam uma ABI pública reutilizável
P2	PTE, VM update, VM flush, BO create/move e correlação por PID	há atividade real de VM/BO	formato completo de page table ou recuperação de fault
P4.5/P4.6/P4.12	CS ioctl, scheduler run job, IB schedule e <code>gfx_0.0.0</code>	há submissão GFX observada	que o workload controlado foi executado e produziu readback
P5.01	exports OpenCL e bibliotecas	existe uma superfície OpenCL vendor	que um dispatch externo foi concluído
P5.04	trace sem eventos de CS/scheduler/IB compute	não houve captura compute demonstrativa	sucesso de compute
P5.05	tracepoints SGPU e AMDGPU enumeráveis	há pontos de observação úteis	que a enumeração seja um teste de hardware

Cadeias comprovadas

```
Device Tree sgpu@22200000
→ G3DCORE power-domain / IRQ / regiões MMIO
→ VM/BO/PTE e flush
→ amdgpu_cs_ioctl
→ amdgpu_sched_run_job
→ amdgpu_ib_schedule em gfx_0.0.0
```

A cadeia acima é observada no caminho vendor/SGPU fornecido. Ela não representa ainda um driver novo.

Cadeias ainda abertas

```
cliente externo → UAPI mínima → BO/VM → IB controlado  
→ fence de conclusão → buffer de saída
```

```
OpenCL externo → contexto → fila → NDRange → readback
```

Essas duas cadeias são objetivos de estudo, não resultados publicados.

Política de publicação

Os P e scripts são material de laboratório. O GitHub recebe a classificação e a síntese, mas não os testes baixáveis.